Guida al deployment online Java/JSP e Spring Boot

Tutorial passo dopo passo: dal repository GitHub a un URL pubblico HTTPS usando Docker e Render. Comprende due percorsi alternativi e una checklist di verifica.

1. Scegliere il percorso

GitHub Pages NON esegue Java, JSP o Servlet lato server. Per pubblicare applicazioni dinamiche serve un runtime Java accessibile online. Render può eseguire un container Docker con Tomcat oppure un JAR Spring Boot.

Percorso A: Java con JSP/Servlet tradizionali (WAR su Tomcat). Percorso B: Spring Boot (JAR eseguibile con server incorporato). I passaggi GitHub e Render sono comuni.

Prerequisiti: account GitHub e Render, Git, JDK compatibile, Maven; Docker Desktop per la prova locale. Verificare:

```
java -version
mvn -version
git --version
docker --version
```

2. Verificare il progetto prima del deployment

Il progetto deve compilare in locale. Dalla cartella con pom.xml eseguire:

```
mvn clean test
mvn clean package
```

Se il progetto ha già un pom.xml, NON sostituirlo: controllare packaging, versione Java, plugin e dipendenze. Il file .war o .jar risultante si trova normalmente in target/.

3A. Percorso A - Java/JSP e Servlet

Per Tomcat 9 utilizzare le API javax.servlet; per Tomcat 10.1 utilizzare jakarta.servlet. Il seguente esempio assume Java 17, Tomcat 10.1 e un WAR prodotto con nome sustainable-webapp.war. Nel pom.xml serve <packaging>war</packaging>. Se il nome del WAR è diverso, aggiornare COPY.

Struttura minima: pom.xml, src/main/java/, src/main/webapp/index.jsp, Dockerfile.

Creare un file Dockerfile nella radice del repository:

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM tomcat:10.1-jdk17-temurin
RUN rm -rf /usr/local/tomcat/webapps/*
COPY --from=build /app/target/sustainable-webapp.war \
    /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

ROOT.war rende il sito raggiungibile alla radice dell'URL. Se l'applicazione usa javax.servlet, scegliere un'immagine Tomcat 9 compatibile e verificare la versione Java.

3B. Percorso B - Spring Boot

Per un progetto Spring Boot con Maven che genera un unico JAR eseguibile e utilizza Java 17, creare questo Dockerfile. Se si utilizza Java 21, modificare entrambe le immagini. Alcuni progetti Spring Boot generano un WAR o richiedono configurazioni diverse.

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar /app/app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

In src/main/resources/application.properties aggiungere, se necessario:

```
server.port=${PORT:8080}
```

Attenzione: il pattern *.jar deve corrispondere a un solo JAR eseguibile. Se la build produce anche altri JAR, specificare il nome esatto. Spring MVC con JSP e packaging WAR può invece richiedere il percorso A.

4. File di esclusione e prova locale

Creare .dockerignore per non inviare alla build file inutili:

```
target/
.git/
.idea/
.vscode/
.env
*.log
```

Avviare Docker Desktop, quindi nella directory del progetto eseguire:

```
docker build -t java-webapp .
docker run --rm -p 8080:8080 java-webapp
```

Aprire http://localhost:8080. Se la porta è occupata, usare -p 8081:8080 e aprire http://localhost:8081. Interrompere con Ctrl+C. Se il container non parte, leggere i messaggi del terminale.

5. Caricare il progetto su GitHub

Creare un repository su github.com. Se il progetto NON è già un repository Git, eseguire:

```
git init
git add .
git commit -m "Prepare deployment"
git branch -M main
git remote add origin https://github.com/USERNAME/REPO.git
git push -u origin main
```

Sostituire USERNAME e REPO. Se il progetto usa già Git, limitarsi a commit e push. Aggiungere target/, .env, file IDE e credenziali a .gitignore. Non pubblicare password, token o chiavi API.

6. Pubblicare online su Render

1) Aprire https://dashboard.render.com/ e accedere. 2) Selezionare New > Web Service. 3) Collegare GitHub e scegliere il repository. 4) Selezionare Docker come linguaggio/runtime. 5) Impostare il branch main e Dockerfile Path ./Dockerfile (se richiesto). 6) Se disponibile, scegliere il piano Free. 7) Configurare PORT=8080 come variabile d'ambiente. 8) Avviare Deploy Web Service.

Render costruisce l'immagine dal Dockerfile e avvia il container. Controllare gli eventi e i log fino a quando il servizio è attivo. Aprire l'URL HTTPS assegnato (tipicamente https://nome-servizio.onrender.com). Il nome

effettivo è stabilito da Render.

Il Dockerfile esposto qui ascolta sulla porta 8080. La porta deve essere coerente con la configurazione del servizio. Per configurazioni avanzate verificare la documentazione Render sulle porte.

7. Verifica e aggiornamenti

Controllare: pagina iniziale; JSP o endpoint dinamici; collegamenti CSS/JS; form; Servlet o controller; assenza di errori 404/500; apertura dell'URL da un altro dispositivo. Per Spring Boot provare anche gli endpoint HTTP definiti dai controller.

Per aggiornare il sito:

```
git add .
git commit -m "Update application"
git push origin main
```

Con Auto-Deploy attivo su Render, un push al branch configurato avvia un nuovo deployment. In caso di errore controllare i log e riprodurre il problema localmente.

8. Database e file persistenti

Se l'applicazione usa MySQL/PostgreSQL, occorre un database remoto raggiungibile dal container. localhost dentro il container NON indica il computer dello studente. Impostare URL JDBC, utente e password con variabili d'ambiente su Render; non inserire segreti nel repository. Valutare limiti e costi del provider database. Il filesystem del Web Service Free non è persistente: non usarlo per salvare file importanti.

9. Problemi frequenti

Build Maven fallita: verificare dipendenze e versione Java con mvn clean package. 404: controllare percorso WAR, nome ROOT.war o mapping controller. 500: leggere i log applicativi. javax/jakarta: allineare Tomcat e Servlet API. Errore database: controllare URL JDBC, firewall e variabili. Avvio lento: il piano Free può sospendere i servizi inattivi.

10. Limiti e consegna

I servizi gratuiti Render sono destinati a prototipi e didattica: possono sospendersi dopo inattività e prevedono limiti di ore, risorse e traffico. Prezzi e condizioni possono cambiare: verificare https://render.com/docs/free prima di creare il servizio. Non utilizzare il piano gratuito per applicazioni che richiedono alta disponibilità.

Documentazione ufficiale

Render Docker: https://render.com/docs/docker
Render Free: https://render.com/docs/free
Tomcat: https://tomcat.apache.org/
Docker: https://docs.docker.com/get-started/
Maven: https://maven.apache.org/guides/getting-started/

Percorso aggiuntivo — Pubblicare un sito statico

Questo percorso è dedicato a siti composti da **HTML, CSS e JavaScript lato browser**, senza un backend Java in esecuzione. Per questo tipo di progetto è possibile usare **GitHub Pages**, senza Docker, Maven o Tomcat.

Passo 1 — Preparare i file

Organizzate il progetto con un file **index.html** nella directory principale (oppure nella cartella **docs**), insieme a eventuali file CSS, JavaScript e immagini.

```
mio-sito/
  index.html
  style.css
  script.js
  images/
```

Esempio minimo di index.html:

```
<!DOCTYPE html>
<html lang="it">
<head><meta charset="UTF-8"><title>Il mio sito</title></head>
<body><h1>Il mio sito e online!</h1></body>
</html>
```

Passo 2 — Caricare il sito su GitHub

Create un nuovo repository su github.com e caricate i file del sito. Se usate Git da terminale, dopo aver configurato il repository remoto eseguite:

```
git add .
git commit -m "Publish static website"
git push origin main
```

Passo 3 — Attivare GitHub Pages

Nel repository aprite **Settings** → **Pages**. In **Build and deployment**, scegliete **Deploy from a branch**, selezionate **main** e **/(root)** (oppure **/docs** se avete messo lì il sito), quindi salvate. Attendete la pubblicazione.

Passo 4 — Aprire e verificare il sito

GitHub mostrerà l'indirizzo pubblico del sito, generalmente **https://USERNAME.github.io/NOME-REPOSITORY/**. Apritelo in un browser e controllate link, immagini, CSS e JavaScript. Per i repository di progetto, ricordate che i percorsi assoluti che iniziano con / potrebbero non funzionare come previsto.

Passo 5 — Aggiornare il sito

Modificate i file, create un nuovo commit e inviatelo su GitHub. GitHub Pages pubblicherà la versione aggiornata dopo il completamento del deployment.

Alternativa — Netlify

Per siti statici potete anche utilizzare **Netlify**: create un account, collegate il repository GitHub, impostate eventualmente il comando di build e la directory di output, quindi pubblicate. Per HTML/CSS/JS semplici non serve un comando di build. Per React o Vue può essere necessario indicare la cartella generata (per esempio dist).

Quale percorso scegliere?

HTML/CSS/JS senza backend: GitHub Pages o Netlify. **Servlet/JSP:** Docker + Tomcat + Render. **Spring Boot:** Docker + Java + Render. GitHub Pages non esegue Servlet, JSP, Spring Boot né codice Java lato server.

Documentazione ufficiale

GitHub Pages: https://docs.github.com/en/pages/quickstart
Netlify: https://docs.netlify.com/start/overview/